InternsElite

AIML with Python Training Program

MAJOR PROJECT REPORT

FAKE NEWS DETECTION USING MACHINE LEARNING

An educational text classifier built with the WELFake dataset, TF-IDF features, and a Streamlit web app

SUBMITTED BY	
Name	**Aditya Rathore**
Student ID	**AIML - A6/SEP-11772**
Institute	**InternsElite**
Trainer	**Pratham Saxena**
Program	**AIML with Python Training Program**
Batch	**September 2026**

Submission date: **[Submission Date]**

This report describes a student project. The classifier is not a fact-checking service, and its predictions may be wrong.

Table of Contents

1. Abstract
2. Introduction
3. Problem Statement
4. Project Objectives
5. Scope of the Project
6. Dataset Description and Source
7. Data Preprocessing
8. Exploratory Data Analysis
9. Feature Extraction with TF-IDF
10. Model Training and Test Split
11. Evaluation Metrics
12. Results and Model Comparison
13. Application Design and Implementation
14. Screenshots and Evidence
15. Challenges and Solutions
16. Limitations
17. Future Improvements
18. Conclusion
19. References

Page numbers are stored as fields. If they look out of date after editing, select all (Ctrl+A) and press F9 to update them.

1. Abstract

Online platforms make it easy for news-style articles to spread quickly, and readers rarely have time to check every article they see. This project explores whether a machine learning model can learn patterns in article text that separate articles labeled Fake from those labeled Real. It is an educational student project and is not a fact-checking service.

The project uses the WELFake dataset, which contains 72,134 labeled articles (35,028 Fake and 37,106 Real). Missing title and text values were filled with empty strings, the title and text were combined into one content field, and whitespace was trimmed. No rows were removed during cleaning. The text was converted into numerical features with TF-IDF (English stop words, up to 20,000 features, unigrams and bigrams, min_df=2). The data was split 80/20 into 57,707 training articles and 14,427 test articles, and three models were compared: Naive Bayes, Logistic Regression, and Linear SVM.

On the held-out test set, Linear SVM achieved the highest accuracy and macro F1 score (both 96.6%), compared with 95.2% for Logistic Regression and 86.8% for Naive Bayes. Linear SVM was therefore used in a Streamlit web app in which a user can enter article text and see a “Likely real” or “Likely fake” prediction. The app also includes a model evaluation dashboard and warns the user when the input is empty or shorter than 100 words.

These results come from one held-out portion of a single dataset. They do not guarantee similar performance on articles from other sources or on real-world news. A prediction is a statistical guess based on text patterns; it can be wrong and is not proof that a story is true or false.

Keywords: fake news, text classification, TF-IDF, Linear SVM, WELFake, Streamlit

Project at a glance

Table 1: *Key facts about the project*

Item	Detail
Task	Classify news article text as Fake (label 0) or Real (label 1)
Dataset	WELFake, 72,134 articles
Features	TF-IDF (English stop words, up to 20,000 features, unigrams and bigrams, min_df=2)
Split	80% training (57,707) and 20% testing (14,427)
Models compared	Naive Bayes, Logistic Regression, Linear SVM
Selected model	Linear SVM (96.6% accuracy and 96.6% macro F1 on the held-out test set)
Application	Streamlit web app with prediction, input warnings, and an evaluation dashboard

Item	Detail
Repository	https://github.com/AdiRathoreX/Fake_news_detection_classifier

2. Introduction

Misinformation is a general challenge in the digital age. Articles, posts, and headlines can be shared widely within minutes, while checking a claim carefully usually takes much longer. Because of this gap, researchers and students are interested in tools that can help flag text that deserves a closer look.

Machine learning offers one way to approach the problem. In supervised text classification, a model is shown many example texts together with their labels. It learns patterns that link the words in a text to the label that was assigned to it, and it can then predict a label for a new text. Before a model can use text, the text has to be converted into numbers; this project uses a standard method called TF-IDF for that step.

It is important to be clear about what such a model does and does not do. A text classifier learns statistical patterns in the wording of the articles in its training data. It does not look up facts, check sources, or understand whether a claim is true. Its output is therefore a prediction about how an article resembles the labeled examples it learned from, not a verdict about the real world.

This report documents a student project that applies these ideas to the WELFake dataset. It compares three classical models, selects the best-performing one on a held-out test set, and presents it in a simple Streamlit web app together with clear warnings about its limits.

3. Problem Statement

Readers meet a very large amount of news-style text, and it is not practical to assess every article manually. The problem addressed in this project is: given the text of a news article, can a machine learning model assign it one of two labels, Fake (0) or Real (1), that agree with the labels in a labeled dataset?

The task is framed as binary text classification. The project asks how well standard text-feature and model choices perform on this task and which of the three tested models performs best on the same held-out test set.

The project does not attempt to determine whether a particular claim is factually correct. The model learns relationships between wording and the dataset's labels, so the result should be read as a pattern-based prediction.

A second part of the problem is usability. A trained model is difficult for non-programmers to try, so the project also provides a web app where a user can enter article text, receive a prediction, and see how the models were evaluated.

4. Project Objectives

The objectives of the project are to:

1. Load and describe the WELFake dataset, including its size, columns, labels, and class counts.
2. Clean the data by removing the unnamed index column, filling missing title and text values, combining title and text, trimming whitespace, and confirming that no empty content remains.
3. Explore the label distribution and the distribution of article lengths.
4. Convert article text into numerical features using TF-IDF.
5. Train Naive Bayes, Logistic Regression, and Linear SVM models and compare them on the same held-out test set using accuracy, macro F1, and confusion matrices.
6. Save the selected Linear SVM model and the TF-IDF vectorizer so that they can be reused by an application.
7. Build a Streamlit app with text input, a word count, warnings for empty and short input, a prediction display, an evaluation dashboard, and a disclaimer.
8. Document the work and communicate clearly that predictions can be wrong and are not proof of truth or falsity.

5. Scope of the Project

The scope of the project is deliberately limited so that its claims stay accurate.

5.1 In scope

- Binary classification of news article text as Fake (0) or Real (1) using the labels supplied with the WELFake dataset.
- TF-IDF text features and a comparison of three classical models: Naive Bayes, Logistic Regression, and Linear SVM.
- Evaluation on a single 80/20 train/test split using accuracy, macro F1, and confusion matrices.
- A Streamlit web app that uses the saved Linear SVM model and TF-IDF vectorizer.

5.2 Out of scope

- Fact-checking: the project does not verify claims against external sources or evidence.
- Identifying the author, publisher, or origin of an article.
- Images, video, or other non-text content.
- Evaluation on other datasets, news sources, or time periods.
- Claims about real-world accuracy. The text pipeline uses an English stop-word list, so the project is intended for English-language article text.

> **Important:** This is a text classifier for a student project, not a fact-checking service. Its predictions may be wrong and do not prove that a story is true or false.

6. Dataset Description and Source

6.1 Source of the dataset

The project uses the WELFake dataset. The official dataset record is available on Zenodo at https://zenodo.org/records/4561253. The dataset contains news articles that are each labeled as Fake or Real.

6.2 Structure and labels

The dataset contains four original columns: `Unnamed: 0`, `title`, `text`, and `label`. The `Unnamed: 0` column is an index column and carries no article information. The `label` column uses 0 for Fake articles and 1 for Real articles.

Table 2: *Summary of the WELFake dataset as used in this project*

Item	Detail
Dataset	WELFake
Official page	https://zenodo.org/records/4561253
Total articles	72,134
Original columns	`Unnamed: 0`, `title`, `text`, `label`
Label encoding	0 = Fake, 1 = Real
Fake articles (0)	35,028 (48.6%)
Real articles (1)	37,106 (51.4%)
Missing values found	558 titles and 39 text fields

The two classes are of similar size: 35,028 Fake and 37,106 Real articles. A screenshot of the dataset overview is shown as Figure 1 in Chapter 14.

7. Data Preprocessing

Before any model could be trained, the raw data was cleaned so that every article had a single, usable block of text. The following steps were applied in order:

1. **Remove the index column.** The `Unnamed: 0` column was dropped because it is only a row index.
2. **Fill missing values.** Missing values were found in 558 title fields and 39 text fields. Each missing value was replaced with an empty string so that no article was lost because one of its two fields was blank.
3. **Combine title and text.** The title and the article text were joined into a single content field, so that the model can use both.
4. **Trim whitespace.** Leading and trailing whitespace was removed from the combined content.
5. **Remove empty content.** Rows with empty content were removed.

Table 3: *Dataset size before and after cleaning*

Check	Result
Rows after cleaning	72,134
Empty content rows after cleaning	0

The number of rows after cleaning (72,134) is the same as the original number of articles, and no empty content rows remained. This is consistent with filling missing values rather than deleting the affected rows. The cleaning step is shown as Figure 2 in Chapter 14.

8. Exploratory Data Analysis

8.1 Label distribution

The dataset contains 35,028 Fake articles (48.6%) and 37,106 Real articles (51.4%). The difference between the two classes is 2,078 articles, so the dataset is close to balanced. This matters because a heavily imbalanced dataset can make accuracy look better than it really is. The label distribution is shown as Figure 3 in Chapter 14.

Table 4: *Label distribution*

Label	Meaning	Articles	Share of dataset
0	Fake	35,028	48.6%
1	Real	37,106	51.4%

8.2 Article length distribution

The length of each article was measured in words. The summary statistics are given in the table below.

Table 5: *Article word-count summary (all 72,134 articles)*

Statistic	Words
Count	72,134
Mean	552.72
Standard deviation	625.66
Minimum	1
25th percentile	239
Median	410
75th percentile	679
Maximum	24,243

The middle half of the articles (from the 25th to the 75th percentile) has between 239 and 679 words. The mean (552.72) is higher than the median (410), and the standard deviation (625.66) is large compared with the mean. Together, these values show that the distribution is skewed to the right: most articles are of moderate length, but a small number of very long articles pull the average upward.

The article-length histogram (Figure 4 in Chapter 14) was displayed over the range of 0 to 1,500 words to make the main distribution readable. The maximum article length is 24,243 words, which is

far outside that range because the dataset includes very long outliers. The minimum is a single word, so the dataset also includes extremely short texts.

These findings explain a design choice in the app: because the typical article in the dataset has hundreds of words, the model is expected to work best with article-length input, and the app warns users when their text is short.

9. Feature Extraction with TF-IDF

9.1 Turning text into numbers

Machine learning models work with numbers, not with words. A common way to represent a text numerically is to build a vocabulary of the words (and short phrases) that appear in the collection of articles, and then describe each article by how strongly each vocabulary item is present in it. This is the idea behind TF-IDF, which stands for term frequency–inverse document frequency.

- **Term frequency (TF)** asks: how often does this word appear in this article? A word that is repeated many times is probably important for that article.
- **Inverse document frequency (IDF)** asks: how rare is this word across all articles? A word that appears in almost every article (such as “the”) tells us little, while a word that appears in only some articles is more informative.
- **TF-IDF** combines the two ideas. A word receives a high weight when it is frequent in one article but not common across the whole collection.

After this step, each article becomes a list of numbers, one for each vocabulary item. Most of these numbers are zero, because any single article uses only a small part of the whole vocabulary. The classification models learn from these lists of numbers and from the known labels.

9.2 Settings used in the notebook

The notebook used the settings below.

Table 6: *TF-IDF settings used in the notebook*

Setting	Value	Plain-language meaning
Stop words	English	Very common English words are ignored because they carry little useful information.
Maximum features	20,000	The vocabulary is capped at 20,000 terms.
N-gram range	Unigrams and bigrams	Both single words and two-word phrases are used as features.
`min_df`	2	A term must appear in at least two articles to be kept.

Using bigrams lets the features capture short phrases as well as single words, which can carry more meaning than either word on its own. The `min_df=2` setting removes terms that appear in only one article, and the 20,000-term cap keeps the feature set at a manageable size.

[Please fill in: confirm from the notebook how the TF-IDF vectorizer was fitted (for example, on the training split only or on the full dataset before splitting) and describe it here.]

The fitted vectorizer was saved as `tfidf_vectorizer.joblib` so that the app can transform new text in exactly the same way as the training text.

10. Model Training and Test Split

10.1 Train/test split

The cleaned dataset was divided into a training set (80%) and a test set (20%). The models learn only from the training articles; the test articles are held back and used only to measure performance on data that the models did not learn from.

Table 7: *Train/test split*

Set	Articles	Share
Training set	57,707	80%
Test set	14,427	20%
Total	72,134	100%

10.2 Models compared

Three classical models were trained on the TF-IDF features and compared on the same held-out test set of 14,427 articles:

- **Naive Bayes** is a simple probabilistic model that estimates how likely each word is under each label and combines these estimates. It is fast and often used as a baseline for text.
- **Logistic Regression** is a linear model that combines the feature weights into a probability for each label.
- **Linear SVM** (support vector machine) is a linear model that looks for a dividing boundary between the two classes with as wide a margin as possible. It often performs well on high-dimensional text features.

The Linear SVM model and the TF-IDF vectorizer were saved as `fake_news_svm_model.joblib` and `tfidf_vectorizer.joblib` and are used by the deployed app.

10.3 Details to confirm

The following details are not part of the facts supplied for this report and should be completed from the notebook or the project files before submission.

Table 8: *Reproducibility details to complete*

Detail	Value
Random seed for the split	**[Please fill in]**
Model hyperparameters	**[Please fill in]**
Python and library versions	**[Please fill in]**

11. Evaluation Metrics

Choosing a model requires measuring how well it predicts labels it has not seen. This chapter explains the measures used in the report in plain language. In the examples, Fake is treated as the "positive" class.

Table 9: *Evaluation measures*

Measure	Plain-language meaning	Formula
Accuracy	Out of all test articles, the share that the model labeled correctly.	(correct predictions) / (all predictions)
Precision	Of the articles the model labeled Fake, the share that really are Fake.	correct Fake / all predicted Fake
Recall	Of the articles that really are Fake, the share the model found.	correct Fake / all actual Fake
F1 score	A single score that balances precision and recall; it is high only when both are high.	2 × precision × recall / (precision + recall)
Macro F1	The F1 score calculated separately for Fake and for Real, then averaged, so both classes count equally.	(F1 of Fake + F1 of Real) / 2

11.1 Confusion matrices

A confusion matrix is a small table that shows exactly where a model is right and where it makes mistakes. In this report, the **rows are the actual labels** and the **columns are the predictions**, in the order Fake, Real. Each matrix therefore has four cells:

- **Top-left:** Fake articles correctly classified as Fake.
- **Top-right:** Fake articles predicted as Real.
- **Bottom-left:** Real articles predicted as Fake.
- **Bottom-right:** Real articles correctly classified as Real.

The top-left and bottom-right cells are the correct predictions; the other two cells are the errors.

11.2 Worked example

For the Linear SVM matrix, 6,716 Fake articles were found correctly, 195 Real articles were wrongly labeled Fake, and 290 Fake articles were missed. Precision for the Fake class is 6,716 / (6,716 + 195) = 97.2%, and recall for the Fake class is 6,716 / (6,716 + 290) = 95.9%. These two example values were calculated from the confusion matrix for illustration.

12. Results and Model Comparison

All three models were evaluated on the same held-out test set of 14,427 articles. The notebook results are summarized in the table below and the full evaluation output is shown as Figure 5 in Chapter 14.

12.1 Accuracy and macro F1

Table 10: *Accuracy and macro F1 on the held-out test set*

Model	Accuracy	Macro F1
Naive Bayes	86.8%	86.8%
Logistic Regression	95.2%	95.2%
Linear SVM	**96.6%**	**96.6%**

12.2 Naive Bayes confusion matrix

Table 11: *Confusion matrix: Naive Bayes (rows = actual, columns = predicted)*

Actual \ Predicted	Predicted: Fake (0)	Predicted: Real (1)
Actual: Fake (0)	**5,925**	**1,081**
Actual: Real (1)	**817**	**6,604**

- **Top-left (5,925):** Fake articles correctly classified as Fake.
- **Top-right (1,081):** Fake articles wrongly predicted as Real.
- **Bottom-left (817):** Real articles wrongly predicted as Fake.
- **Bottom-right (6,604):** Real articles correctly classified as Real.

The test set therefore contained 7,006 Fake and 7,421 Real articles. Naive Bayes classified 12,529 of the 14,427 test articles correctly (86.8%) and made 1,898 errors.

12.3 Logistic Regression confusion matrix

Table 12: *Confusion matrix: Logistic Regression (rows = actual, columns = predicted)*

Actual \ Predicted	Predicted: Fake (0)	Predicted: Real (1)
Actual: Fake (0)	**6,603**	**403**
Actual: Real (1)	**291**	**7,130**

- **Top-left (6,603):** Fake articles correctly classified as Fake.

- **Top-right (403):** Fake articles wrongly predicted as Real.
- **Bottom-left (291):** Real articles wrongly predicted as Fake.
- **Bottom-right (7,130):** Real articles correctly classified as Real.

The test set therefore contained 7,006 Fake and 7,421 Real articles. Logistic Regression classified 13,733 of the 14,427 test articles correctly (95.2%) and made 694 errors.

12.4 Linear SVM confusion matrix

Table 13: *Confusion matrix: Linear SVM (rows = actual, columns = predicted)*

Actual \ Predicted	Predicted: Fake (0)	Predicted: Real (1)
Actual: Fake (0)	**6,716**	**290**
Actual: Real (1)	**195**	**7,226**

- **Top-left (6,716):** Fake articles correctly classified as Fake.
- **Top-right (290):** Fake articles wrongly predicted as Real.
- **Bottom-left (195):** Real articles wrongly predicted as Fake.
- **Bottom-right (7,226):** Real articles correctly classified as Real.

The test set therefore contained 7,006 Fake and 7,421 Real articles. Linear SVM classified 13,942 of the 14,427 test articles correctly (96.6%) and made 485 errors.

12.5 Errors compared across models

The table below summarizes the errors. These values were calculated from the confusion matrices above; they are not separate measurements.

Table 14: *Errors and per-class recall (calculated from the confusion matrices)*

Model	Misclassified	Error rate	Fake recall	Real recall
Naive Bayes	1,898 of 14,427	13.2%	84.6%	89.0%
Logistic Regression	694 of 14,427	4.8%	94.2%	96.1%
Linear SVM	**485 of 14,427**	**3.4%**	**95.9%**	**97.4%**

12.6 Interpretation

Linear SVM had the highest accuracy and macro F1 among the three tested models on this held-out dataset (96.6% for both). Logistic Regression followed with 95.2%, and Naive Bayes was lowest with 86.8%. Compared with Naive Bayes, the two linear models made far fewer errors: 1,898 for Naive Bayes, 694 for Logistic Regression, and 485 for Linear SVM. Linear SVM was therefore selected for the app.

In all three matrices, more Fake articles were predicted as Real than Real articles were predicted as Fake (for Linear SVM, 290 compared with 195). This means that a Fake article was somewhat more likely to be missed than a Real article was to be wrongly flagged.

> **Please note:** A high score on this test set does not guarantee equivalent performance on articles from new sources or on real-world news. The results describe how the models behaved on one held-out portion of the WELFake dataset.

13. Application Design and Implementation

To make the model easy to try, the project includes a web application built with Streamlit. The app loads the saved Linear SVM model and the saved TF-IDF vectorizer and lets a user test the classifier on their own text.

13.1 Workflow

The flow from input to result is shown below.



13.2 Features

Table 15: *Features of the Streamlit app*

Feature	Behavior
Text input	A text area where the user enters news article text.
Word count	The app displays the number of words in the input.
Empty-input warning	If the input is empty, the app shows a warning.
Short-input warning	If the input has fewer than 100 words, the app warns that the model was trained mostly on full articles and that short text may produce unreliable guesses.
Prediction	The saved TF-IDF vectorizer and the saved Linear SVM model are used to make a prediction, which is displayed as “Likely real” or “Likely fake”.
Evaluation dashboard	Shows accuracy comparisons, a side-by-side accuracy and macro F1 chart, and confusion matrices for the three models.
Disclaimer	States that predictions may be wrong and are not proof of truth or falsity.

13.3 Saved artifacts

The app depends on two saved files, `fake_news_svm_model.joblib` (the trained Linear SVM model) and `tfidf_vectorizer.joblib` (the TF-IDF vectorizer). Using the saved vectorizer ensures that new text is converted to numbers in the same way as the text the model was trained on. The wording “Likely real” and “Likely fake” is intentional: it signals that the output is a prediction rather than a fact.

13.4 Dashboard and disclaimer

The dashboard lets the user see how the three models compared during evaluation, so the choice of Linear SVM is transparent. It repeats the accuracy and macro F1 values reported in Chapter 12 and the three confusion matrices. The disclaimer is displayed in the app so that users do not mistake a prediction for a fact-check.

13.5 Running the app locally

According to the project README, `app.py`, both `.joblib` files, and `requirements.txt` should be kept in the same folder. In a Windows terminal opened in that folder, the app is started with:

```
py -m pip install -r requirements.txt
py -m streamlit run app.py
```

13.6 Links

- **Repository:** https://github.com/AdiRathoreX/Fake_news_detection_classifier
- **Live app:** **[Paste the live Streamlit app URL]**

> **Disclaimer:** The app's predictions may be wrong. A prediction is not proof that a story is true or false, and the app is not a fact-checking service.

14. Screenshots and Evidence

This chapter collects the screenshots that document the notebook work and the deployed app. Each figure is referred to from the chapter that it supports.

14.1 Notebook evidence

Figure 1 shows the dataset overview from the notebook, including the dataset dimensions, the column names, and the counts of Fake and Real labels. It supports the dataset description in Chapter 6.

IMAGE PLACEHOLDER

Insert the screenshot file **`Dataset Overview.png`** here.

Figure 1: *Dataset overview: dataset dimensions, columns, and label counts.*

Figure 2 shows the cleaning step in which missing values are handled and the title and article text are combined into a single content field. It supports Chapter 7.

IMAGE PLACEHOLDER

Insert the screenshot file **`Text Preprocessing.png`** here.

Figure 2: *Text preprocessing: cleaning and combining the title and article text.*

Figure 3 shows how the Fake and Real labels are distributed across the dataset. It supports the label analysis in Chapter 8.

IMAGE PLACEHOLDER
Insert the screenshot file **`Label Distribution.png`** here.

Figure 3: *Label distribution: counts of Fake and Real articles.*

Figure 4 shows the article word-count distribution. The histogram is displayed over the range of 0 to 1,500 words so that the main part of the distribution is readable; a small number of very long articles lie beyond this range. It supports Chapter 8.

IMAGE PLACEHOLDER
Insert the screenshot file **`Article Length Distribution.png`** here.

Figure 4: *Article length distribution: histogram of article word counts.*

Figure 5 shows the model classification metrics and confusion matrices produced in the notebook for the three models on the held-out test set. It supports Chapters 11 and 12.

IMAGE PLACEHOLDER
Insert the screenshot file **`Model Evaluation.png`** here.

Figure 5: *Model evaluation: classification metrics and confusion matrices from the notebook.*

14.2 Application evidence

Figure 6 shows the model evaluation dashboard inside the deployed app, which presents the accuracy comparison, a side-by-side accuracy and macro F1 chart, and the confusion matrices for the three models. It supports Chapter 13.

IMAGE PLACEHOLDER
Insert the screenshot file `App Model Dashboard.png` here.

Figure 6: *Streamlit app model dashboard: model metrics, comparison chart, and confusion matrices.*

Figure 7 shows an example article entered into the app together with the prediction returned by the Linear SVM model. The prediction is a statistical guess and is not proof that an article is true or false.

IMAGE PLACEHOLDER
Insert the screenshot file `App Prediction.png` here.

Figure 7: *Streamlit app prediction: example article input and the model's prediction.*

Figure 8 shows how the app responds when it is submitted with empty input.

IMAGE PLACEHOLDER
Insert the screenshot file `Empty Input Warning.png` here.

Figure 8: *Streamlit app empty-input warning: response when the app is submitted with no text.*

15. Challenges and Solutions

The challenges below come directly from the steps of the project work described in this report.

Table 16: *Challenges and how they were handled*

Challenge	How it was handled
Missing text in the dataset (558 titles and 39 text fields)	Missing values were filled with empty strings, title and text were combined, whitespace was trimmed, and a check confirmed that 0 rows with empty content remained, so no articles were lost.
Converting text into numbers a model can use	Text was converted with TF-IDF using English stop words, up to 20,000 features, unigrams and bigrams, and min_df=2.
Comparing models fairly	All three models were evaluated on the same 14,427-article held-out test set using accuracy, macro F1, and confusion matrices.
Very long articles distorting the length plot (maximum 24,243 words)	The histogram was displayed over 0–1,500 words so that the main distribution is readable, and the long outliers are explained in the text.
Communicating the limits of the predictions	The app warns on empty and short input, uses the wording “Likely real” and “Likely fake”, and shows a disclaimer; the report repeats that the classifier is not a fact-checking service.

16. Limitations

- **Not a fact-checker.** The model learns patterns in text. It does not verify claims, check sources, or understand the world, so a prediction is not proof that a story is true or false.
- **One dataset, one split.** The reported scores come from a single 80/20 split of the WELFake dataset. They do not guarantee similar performance on other sources, topics, writing styles, or time periods.
- **Remaining errors.** Even the best model, Linear SVM, misclassified 485 of 14,427 test articles (3.4%): 290 Fake articles were predicted as Real and 195 Real articles were predicted as Fake.
- **Dependence on the dataset's labels.** The model learns from the labels supplied with the dataset. Any mistakes or biases in those labels can carry over into the model.
- **Pattern-based predictions.** The model can only respond to the words and phrases in the text. It may rely on wording, style, or topic patterns in the training data that do not reflect whether a story is true.
- **Short text.** The model was trained mostly on full articles (the median article has 410 words), so short input may produce unreliable guesses. The app warns for fewer than 100 words.
- **Text only and English-oriented.** The model uses article text only; it does not use images, source information, or publication details. The stop-word list is English.
- **Two classes only.** Articles are labeled Fake or Real; the model has no category for satire, opinion, or mixed content.
- **Details still to confirm.** The random seed, model hyperparameters, and software versions are not stated in this report (see Chapter 10).

17. Future Improvements

The following ideas could be explored in future work:

- Test the saved model on independent datasets and on articles from other sources to see how well the results hold outside WELFake.
- Use cross-validation, and document the random seed and hyperparameters, to make the comparison more robust and reproducible.
- Perform an error analysis of the misclassified articles to understand which kinds of text are hardest for the model.
- Show which words and phrases influenced a prediction, to make the app's output easier to interpret.
- Compare other text representations and other models, and report precision, recall, and F1 for each class.
- Improve the handling of short text, for example by evaluating the model on short articles separately.
- Include additional information such as the source or publication details, where such data is available.
- Add support for other languages and for more than two categories.

18. Conclusion

This project built and evaluated a machine learning text classifier that labels news article text as Fake (0) or Real (1). Using the WELFake dataset of 72,134 articles, the data was cleaned, the title and text were combined, and TF-IDF features were created with English stop words, up to 20,000 features, unigrams and bigrams, and min_df=2.

Three models were compared on the same held-out test set of 14,427 articles. Linear SVM achieved the highest accuracy and macro F1 score (96.6%), ahead of Logistic Regression (95.2%) and Naive Bayes (86.8%), and it was selected for a Streamlit web app. The app provides a word count, warnings for empty and short input, a “Likely real” or “Likely fake” prediction, an evaluation dashboard, and a disclaimer.

The project shows that a classical text-classification pipeline can perform well on a labeled dataset. It also shows why careful wording matters: the model has been tested on one dataset only, it makes errors, and its output is a prediction about text patterns. It is a student project, not a fact-checking service, and its predictions should never be treated as proof that a story is true or false.

19. References

[1] WELFake dataset. Zenodo record. https://zenodo.org/records/4561253

[2] Fake_news_detection_classifier (project repository). GitHub. https://github.com/AdiRathoreX/Fake_news_detection_classifier

[3] Streamlit documentation. https://docs.streamlit.io/

[4] Live Streamlit app: **[Paste the live Streamlit app URL]**

[Please fill in: add access dates if your institute's referencing style requires them.]